NovaMind – Complete Project Documentation

A private document and voice assistant built with RAG (Retrieval Augmented Generation). Inspired by the structure of the "Local RAG & Voice Assistant" project at github.com/conbott/JARVIS, rebuilt as a simpler, fully commented version.

1. What NovaMind Is

NovaMind lets you **chat with your own documents** (PDF, TXT, MD), fall back to **live web search** when the documents do not have the answer, and use **voice** to talk to it. The AI model, the embeddings and the database all run on your own computer through **Ollama**, so there is no API cost and your files never leave your machine. (Only the optional voice playback and the web search use the internet.)

The problem it solves. A language model only knows what it was trained on. It has never seen your PDF. RAG fixes this: before asking the model, we *find the relevant passages* in your files and paste them into the prompt. The model then answers from those passages instead of guessing.

Three modes (buttons at the top of the chat):

Mode	Behaviour
Auto	Chooses for you. Small talk goes straight to the AI. Questions go to your documents first; if nothing relevant is found, it searches the web.
Documents	Answers only from your files. If the answer is not there it says so instead of inventing one.
Web	Searches DuckDuckGo and answers from the results, showing source links.

2. Technology Stack and Why Each Piece Was Chosen

Technology	Role	Why this choice
Python 3.10+	Backend language	Best ecosystem for AI work
FastAPI + Uvicorn	Web server / API	Fast, automatic API docs at /docs, built-in support for streaming
Ollama	Runs the LLM and the embedding model locally	Free, offline, one command to download a model
ChromaDB	Vector database	Stores text with its meaning-vector; saves to disk; no server to install
pypdf	Reads PDF text page by page	Pure Python, easy to install
httpx	Calls Ollama's HTTP API	Supports streaming responses
ddgs (DuckDuckGo)	Web search	No API key required
Edge-TTS	Text-to-speech	Natural voices, no GPU
faster-whisper (optional)	Speech-to-text	Runs on CPU, offline
Plain HTML / CSS / JavaScript	User interface	No build step, so nothing extra to install or break

3. Folder Structure

```
novamind/
├── backend/
|   ├── main.py                 # starts the server, plugs everything
together
|   ├── config.py               # all settings in one place
|   ├── requirements.txt        # Python packages needed
|   ├── requirements-voice.txt  # OPTIONAL packages for voice input
|   ├── data/                   # your uploaded files are saved here
|   ├── chroma_db/              # (created automatically) the vector
database
```

```
|   ├── routers/                # the API "doors" the browser knocks on
|   |   ├── chat.py
|   |   ├── documents.py
|   |   ├── system.py
|   |   └── voice.py
|   └── services/               # the real work happens here
|       ├── ollama_client.py
|       ├── vector_store.py
|       ├── ingestion.py
|       ├── query_router.py
|       ├── rag_service.py
|       ├── web_search.py
|       └── voice_service.py
├── frontend/
|   ├── index.html
|   ├── style.css
|   └── app.js
├── .env.example
├── setup.bat / setup.sh
├── start.bat / start.sh
└── README.md
```

Design rule used everywhere: `routers/` only receive requests and send responses. `services/` contain the logic. This keeps each file small and easy to understand.

4. Purpose of Every File and Why We Created It

Root files

- **`.env.example`** – A template of every setting (model names, thresholds, voice options). *Why:* settings should live outside the code so you can change a model without editing Python. You copy it to `backend/.env`; `setup.bat` does this automatically.
- **`setup.bat`** / **`setup.sh`** – One-time installer. It creates `backend/.env`, creates a private Python environment (`.venv`) and installs all packages. *Why:* a virtual environment keeps this project's packages separate from the rest of your computer, which avoids version conflicts.
- **`start.bat`** / **`start.sh`** – Starts the server and (on Windows) opens the browser. *Why:* one double-click instead of remembering a long command.
- **`README.md`** – Short quick-start for anyone opening the folder.

Backend core

- **`backend/main.py`** – The entry point. It creates the FastAPI app, registers the four routers, and serves the `frontend/` folder as the website. *Why:* serving the page and the API from the same server means you run one command and avoid browser cross-origin (CORS) problems. The static-files line must stay last because it matches every URL not already claimed by an API route.
- **`backend/config.py`** – Reads `.env` and exposes one shared `settings` object. *Why:* every other file imports from here, so a setting is defined once. `LLM_MODEL` is stored on this object so it can be switched while the app is running.
- **`backend/requirements.txt`** – Lists the packages `pip install` must fetch. **`requirements-voice.txt`** is separate because `faster-whisper` is large and only needed for voice input.

Backend services (the logic)

- **`services/ollama_client.py`** – The only file that talks to Ollama. Three jobs: `embed()` (text to numbers), `chat_stream()` (get the answer piece by piece), `list_models()`. *Why:* if you ever replace Ollama with another AI provider, this is the only file that changes.
- **`services/vector_store.py`** – Everything about ChromaDB: add chunks, search by similarity, list files, delete a file. *Why:* isolates the database so the rest of the code never touches Chroma directly.
- **`services/ingestion.py`** – Turns an uploaded file into searchable knowledge: read pages, cut into chunks, embed, store. *Why:* models cannot read a whole book at once, so documents must be pre-processed into small pieces.
- **`services/query_router.py`** – Classifies a message as casual, summary, web or specific using simple rules. *Why:* searching documents for "hi" is wasteful and gives odd answers; classification chooses the cheapest correct path.
- **`services/rag_service.py`** – The two retrieval strategies (similarity and spread) plus the prompt builder. *Why:* this is the heart of RAG, so it deserves its own file.
- **`services/web_search.py`** – DuckDuckGo search that never crashes the app if the network or the search engine fails.
- **`services/voice_service.py`** – Text-to-speech and speech-to-text. Heavy libraries are imported *inside* the functions so that the app still starts if voice packages are missing.

Backend routers (the API)

- **`routers/chat.py`** – `POST /api/chat`. The conductor: classify, retrieve, build prompt, stream the answer.
- **`routers/documents.py`** – Upload, list and delete documents. Validates file type and size.
- **`routers/system.py`** – Health check, installed models, switch model, hardware advice.
- **`routers/voice.py`** – `/api/voice/speak` and `/api/voice/transcribe`.

Frontend

- **`frontend/index.html`** – The structure: sidebar (documents, model), chat area, mode buttons, input box.
- **`frontend/style.css`** – The appearance. All colours are variables at the top of the file.
- **`frontend/app.js`** – The behaviour: uploads files, sends messages, reads the streamed reply, records the microphone, plays speech.

5. Step-by-Step Guide to Run NovaMind

Windows

1. **Install Python 3.10 or newer** from python.org. On the first installer screen, tick **"Add Python to PATH"**.
2. **Install Ollama** from ollama.com and let it finish. It runs in the background (system tray).
3. **Unzip** `NovaMind_project.zip` anywhere, for example your Desktop.
4. **Double-click `setup.bat`.** Wait until it prints "Setup finished". This downloads about 300–600 MB of packages the first time.
5. **Download the two AI models.** Open Command Prompt and run:

   ```
   ollama pull llama3.2:3b
   ollama pull nomic-embed-text
   ```

   The first is the chat model (about 2 GB), the second is the embedding model (about 270 MB).
6. **Double-click `start.bat`.** The browser opens at `http://127.0.0.1:8000`. Keep the black window open.

7. **Use it:** click *Add PDF or text file*, wait for "Added …", then ask a question.
8. **Stop:** close the black window.

macOS / Linux

```
bash setup.sh
ollama pull llama3.2:3b
ollama pull nomic-embed-text
bash start.sh
```

Then open `http://127.0.0.1:8000`.

Manual way (any system, if you prefer to see each step)

```
cd novamind/backend
python -m venv .venv
.venv\Scripts\activate          (Windows)   |   source .venv/bin/activate
(Mac/Linux)
pip install -r requirements.txt
copy ..\.env.example .env       (Windows)   |   cp ../.env.example .env
(Mac/Linux)
python -m uvicorn main:app --host 127.0.0.1 --port 8000
```

Enabling voice input (optional)

1. In the activated environment run `pip install -r requirements-voice.txt`.
2. In `backend/.env` set `STT_ENABLED=true`.
3. Restart. Click **Mic**, speak, click **Stop**. The first use downloads the Whisper model.
4. Tick **Read answers aloud** for spoken replies (needs internet).

6. How Data Is Taken In and Processed

There are two journeys: **putting a document in** (ingestion) and **asking a question** (retrieval and generation).

6.1 Ingestion – from file to searchable knowledge

```
Upload (browser)
   → routers/documents.py   validates name, type, size; saves to
backend/data/
   → services/ingestion.py  load_pages(): text + page number
   → split_text():          800-character chunks, 150 overlap
   → ollama_client.embed(): every chunk becomes a list of numbers
   → vector_store.add_chunks(): saved in ChromaDB with source, page, position
```

Step A – Read. `load_pages()` uses `pypdf` to extract text per page. Page numbers start at 1 so answers can cite "page 4". Text files become a single page.

```
reader = PdfReader(str(path))
for number, page in enumerate(reader.pages, start=1):
    text = page.extract_text() or ""      # scanned pages return None
```

Step B – Chunk. A model has a limited context window and search works best on focused passages, so text is cut into pieces of about 800 characters. Consecutive pieces overlap by 150 characters so a sentence on a boundary still appears whole somewhere. The splitter prefers to cut at a full stop or space, and the loop always moves forward so it cannot freeze.

```
while start < length:
    end = min(start + chunk_size, length)
    ...                                   # look back for ". " or " " to cut
cleanly
    start = max(end - overlap, start + 1) # step back by overlap, but always
progress
```

Step C – Embed. An embedding model converts each chunk into a vector (for `nomic-embed-text`, 768 numbers). Chunks with similar meaning get similar vectors. Chunks are sent to Ollama in batches of 32.

Step D – Store. ChromaDB saves the chunk text, its vector and metadata (`source`, `page`, `chunk_index`). Before saving, old chunks of the same filename are deleted, so re-uploading never creates duplicates. The collection uses **cosine distance**, which compares the *direction* of two vectors (meaning) rather than their length.

6.2 Asking a question

```
Browser → POST /api/chat {message, mode, history}
   → query_router.classify()      casual | summary | web | specific
   → choose route                 none | docs | web
   → retrieval                    similarity or spread
   → rag_service.build_messages() system rules + context + question
   → ollama_client.chat_stream()  tokens
   → SSE events                   status, sources, token, done
   → app.js draws the answer word by word
```

Step 1 – Classify (`query_router.py`). Plain regular expressions and keyword lists decide the intent:

Intent	Example	Route in Auto mode
casual	"hi", "thanks"	Straight to the AI, no search
summary	"summarize this document"	Spread retrieval from documents
web	"latest news on ISRO"	Web search
specific	"What is the refund policy?"	Similarity retrieval; web if nothing relevant

This is deliberately not another AI call: it is instant, free and easy to change.

Step 2a – Similarity retrieval (specific questions). The question is embedded with the *same* model as the chunks. ChromaDB returns the 4 closest chunks with a distance; we convert it to a score:

```
score = 1 - distance          # 1.0 = identical meaning, 0 = unrelated
```

Only chunks with `score >= 0.35` (`SIMILARITY_THRESHOLD`) are used. If none pass, Auto mode switches to web search and Documents mode refuses politely. This threshold is the main **anti-hallucination guard**.

Step 2b – Spread retrieval (summaries). Similarity search for "summarize" would return four chunks from the same spot. Instead we:

1. Load all chunks in reading order.
2. Remove reference-list chunks with `looks_like_references()` (starts with "References", or many `[1]`, "et al", "arxiv", "doi:").

3. Pick 8 chunks at evenly spaced positions, always including the first (abstract/introduction) and the last (conclusion).

```
indexes = sorted({round(i * last / (max_chunks - 1)) for i in
range(max_chunks)})
```

Step 2c – Web search. `web_search()` returns up to 5 `{title, url, snippet}` results. If the search fails, an empty list is returned and the app carries on.

Step 3 – Build the prompt (`build_messages`). Three kinds of system instruction exist:

- *docs*: "Use ONLY the document excerpts below… If they do not contain the answer, say exactly 'I could not find that in your documents.'"
- *web*: "Use the web search results below and mention the source titles."
- *none*: plain assistant.

Only the last 6 conversation turns are included and the context is capped at 6000 characters, so the prompt always fits the model's `num_ctx` window of 4096 tokens.

Step 4 – Stream. `chat_stream()` posts to Ollama with `stream: True`. Ollama answers with one JSON line per few characters:

```
{"message": {"content": "Hel"}, "done": false}
{"message": {"content": "lo"},  "done": false}
{"done": true}
```

Each piece is `yield`ed immediately.

6.3 Server-Sent Events (SSE) – how the answer reaches the screen

The server keeps the HTTP connection open and sends small text messages. A blank line ends each message:

```
event: status
data: {"text": "Searching your documents..."}

event: sources
data: {"sources": [{"type": "doc", "title": "report.pdf (page 3)", "score":
0.71}]}

event: token
data: {"text": "The refund"}

event: done
data: {}
```

Event	Meaning	What the browser does
status	Progress text	Shows "Searching…" above the answer
sources	Where the context came from	Lists pages or clickable web links below the answer
token	Next piece of the answer	Appends to the answer bubble
done	Finished	Re-enables the Send button
error	Something failed	Shows the message in red

In `app.js`, network chunks do not line up with messages, so the code keeps a `buffer` and cuts out a message every time it finds `\n\n`:

```
buffer += decoder.decode(value, { stream: true });
while ((end = buffer.indexOf("\n\n")) !== -1) {
  const raw = buffer.slice(0, end);
  buffer = buffer.slice(end + 2);
  ...
}
```

6.4 Voice pipeline

- **Voice in:** the browser's `MediaRecorder` records microphone audio → `POST /api/voice/transcribe` → `faster-whisper` converts it to text on the CPU → the text is sent as a normal chat message.

- **Voice out:** after an answer finishes, if *Read answers aloud* is ticked, `POST /api/voice/speak` → Edge-TTS returns MP3 → the browser plays it.

7. Important Code Points to Understand

1. **Same embedding model everywhere.** Vectors from different models cannot be compared. If you change `EMBEDDING_MODEL`, delete `backend/chroma_db` and upload your files again.
2. **Cosine distance vs similarity.** Chroma returns distance; the code converts with `1 - distance`.
3. **Plain `def` for slow endpoints.** Upload and transcribe are normal `def` functions, so FastAPI runs them in a worker thread and the server stays responsive. Only `speak` is `async` because Edge-TTS is asynchronous.
4. **Generators for streaming.** `run_chat()` uses `yield`, so each event is sent the instant it is created.
5. **Lazy imports for voice.** Heavy packages load only when used, so a missing optional package never prevents startup.
6. **Metadata must be simple types.** ChromaDB accepts only str/int/float/bool in metadata; the code converts page and index to `int`.
7. **Asking for more results than exist raises an error**, so `search()` uses `min(k, total)`.
8. **The static-files mount in `main.py` must be last.**

8. Configuration Reference (`backend/.env`)

Variable	Default	Meaning
`OLLAMA_BASE_URL`	`http://127.0.0.1:11434`	Where Ollama is running
`LLM_MODEL`	`llama3.2:3b`	Chat model (switchable in the UI)
`EMBEDDING_MODEL`	`nomic-embed-text`	Model that creates vectors
`OLLAMA_NUM_CTX`	`4096`	Context window in tokens

Variable	Default	Meaning
`SIMILARITY_THRESHOLD`	`0.35`	Minimum score to treat a chunk as relevant. Raise for stricter answers, lower to find more
`CHUNK_SIZE` / `CHUNK_OVERLAP`	`800` / `150`	Chunking parameters
`MAX_UPLOAD_MB`	`20`	Upload size limit
`STT_ENABLED`	`false`	Turn on voice input
`WHISPER_MODEL`	`base`	Whisper size: tiny, base, small…
`TTS_VOICE`	`en-IN-NeerjaNeural`	Edge-TTS voice name

Changing `CHUNK_SIZE`, `CHUNK_OVERLAP` or `EMBEDDING_MODEL` only affects files uploaded afterwards.

9. API Reference

With the server running, an interactive version is at `http://127.0.0.1:8000/docs`.

Method and path	Purpose
`GET /api/health`	Ollama status, active models, number of stored chunks
`GET /api/models`	Installed Ollama models and the active one
`POST /api/models/select`	Switch the chat model, body `{"llm_model": "name"}`
`GET /api/hardware`	GPU detection and model recommendation
`POST /api/documents/upload`	Upload a file (form field `file`)
`GET /api/documents`	List uploaded files with chunk counts
`DELETE /api/documents/{name}`	Remove a file
`POST /api/chat`	Streamed chat, body `{message, mode, history}`
`POST /api/voice/speak`	Text to MP3

Method and path	Purpose
`POST /api/voice/transcribe`	Audio to text

10. Security and Reliability Decisions

- **Filename sanitising:** `Path(file.filename).name` strips folder parts, blocking attacks such as `../../file.pdf`.
- **File type and size limits** are enforced before anything is processed.
- **No HTML injection:** the browser code uses `textContent` for all user and AI text, never `innerHTML` with dynamic content.
- **Safe links:** only `http://` and `https://` source links are made clickable.
- **Graceful failures:** Ollama offline, missing model, empty PDF, blocked web search and missing voice packages each produce a clear message instead of a crash.
- **Local by default:** the server binds to `127.0.0.1`, so other devices on your network cannot reach it.

11. How This Differs From the Original JARVIS Repository

The original uses a React + TypeScript + Tailwind frontend, LangChain loaders, optional Pinecone cloud storage, Gemini/OpenAI providers, Docker and a rate limiter. NovaMind keeps the same core architecture (hybrid routing, similarity and spread retrieval, web fallback, SSE streaming, voice) but simplifies it so every line can be understood and the install is far less likely to fail:

Original	NovaMind
React 19 + Vite + Tailwind (needs Node.js)	Plain HTML/CSS/JS served by FastAPI
LangChain loaders and splitter	Small custom functions using `pypdf`
Pinecone cloud option, Gemini/OpenAI	Local only: Ollama + ChromaDB
Docker, rate limiting	Left out; can be added later
`qwen2.5-coder:7b` default	`llama3.2:3b`, lighter for ordinary laptops

12. Troubleshooting

Symptom	Cause and fix
Red box: "Ollama is not running"	Start Ollama (`ollama serve`, or open the Ollama app)
"Model … is not installed"	Run the `ollama pull` command shown in the message
"python is not recognised"	Reinstall Python and tick *Add Python to PATH*
Upload says "No readable text found"	The PDF is scanned images. Use an OCR tool first
Upload takes very long	Normal for big PDFs: every chunk is embedded. Try a smaller file first
Answers ignore my document	Use *Documents* mode; lower `SIMILARITY_THRESHOLD` slightly (for example 0.25)
Web mode returns nothing	DuckDuckGo may be rate-limiting or you are offline. Wait a minute
Mic does nothing	Use `http://127.0.0.1:8000` (not a network address), allow the microphone, and set `STT_ENABLED=true`
Port 8000 already in use	Change `--port 8000` in the start script and open the new port
Strange results after changing embedding model	Delete `backend/chroma_db` and re-upload files
Slow answers	Use a smaller model, close other programs; Ollama is faster with a GPU

13. Ideas to Extend the Project

- Add OCR for scanned PDFs (for example with `pytesseract`).
- Show a chat history list and save conversations to disk.
- Add support for `.docx` files with `python-docx`.

- Add a re-ranking step to improve which chunks are chosen.
- Add login and rate limiting before exposing it beyond your own computer.
- Package with Docker for one-command deployment.